Copyright Notice

These slides are distributed under the Creative Commons License.

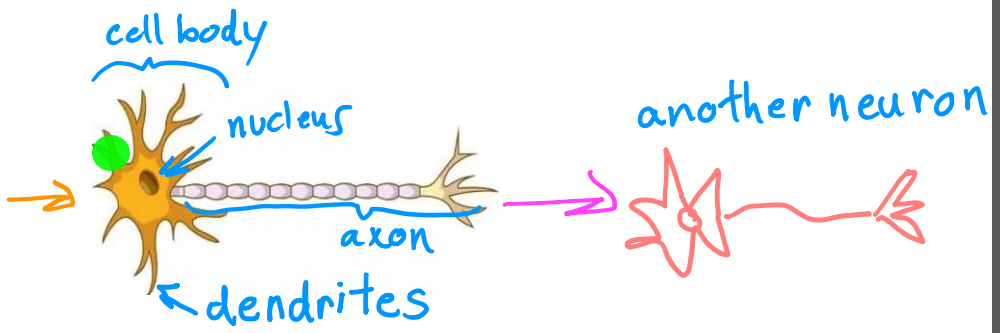
DeepLearning.AI makes these slides available for educational purposes. You may not use or distribute these slides for commercial purposes. You may make copies of these slides and use or distribute them for educational purposes as long as you cite DeepLearning.AI as the source of the slides.

For the rest of the details of the license, see https://creativecommons.org/licenses/by-sa/2.0/legalcode

Neural Networks

Biological neuron

inputs outputs



Simplified mathematical model of a neuron

inputs outputs

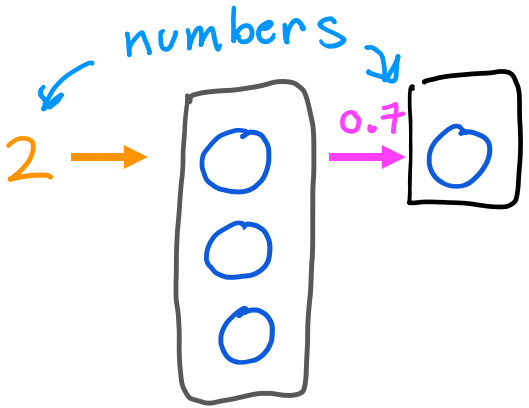
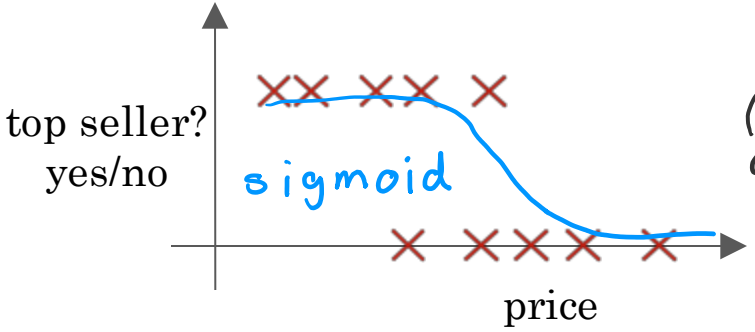


image source: https://biologydictionary.net/sensory-neuron/

Andrew Ng

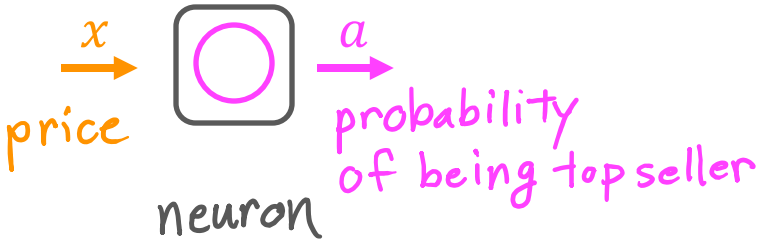
Demand Prediction



$x = price$ input

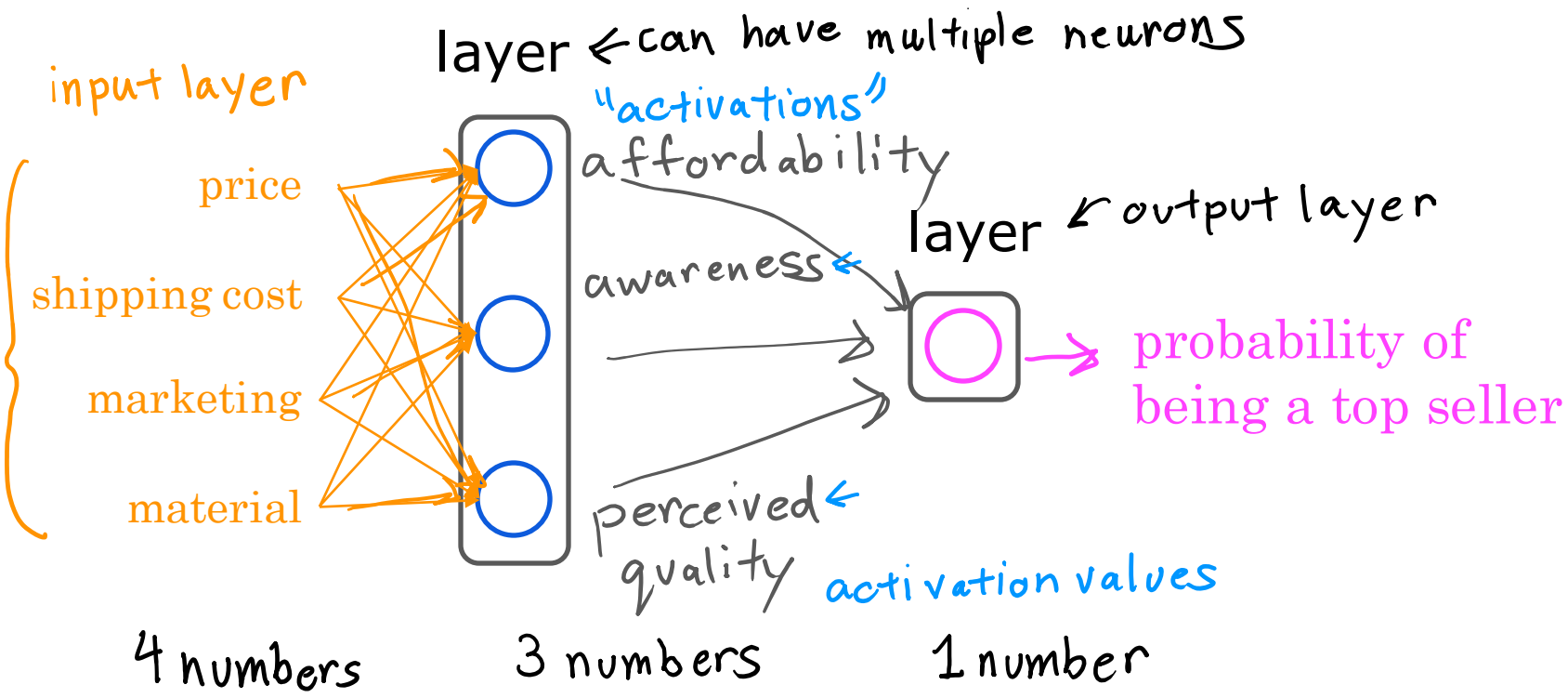
$a = f(x) = \dfrac{1}{1 + e^{-(wx+b)}}$ output

activation



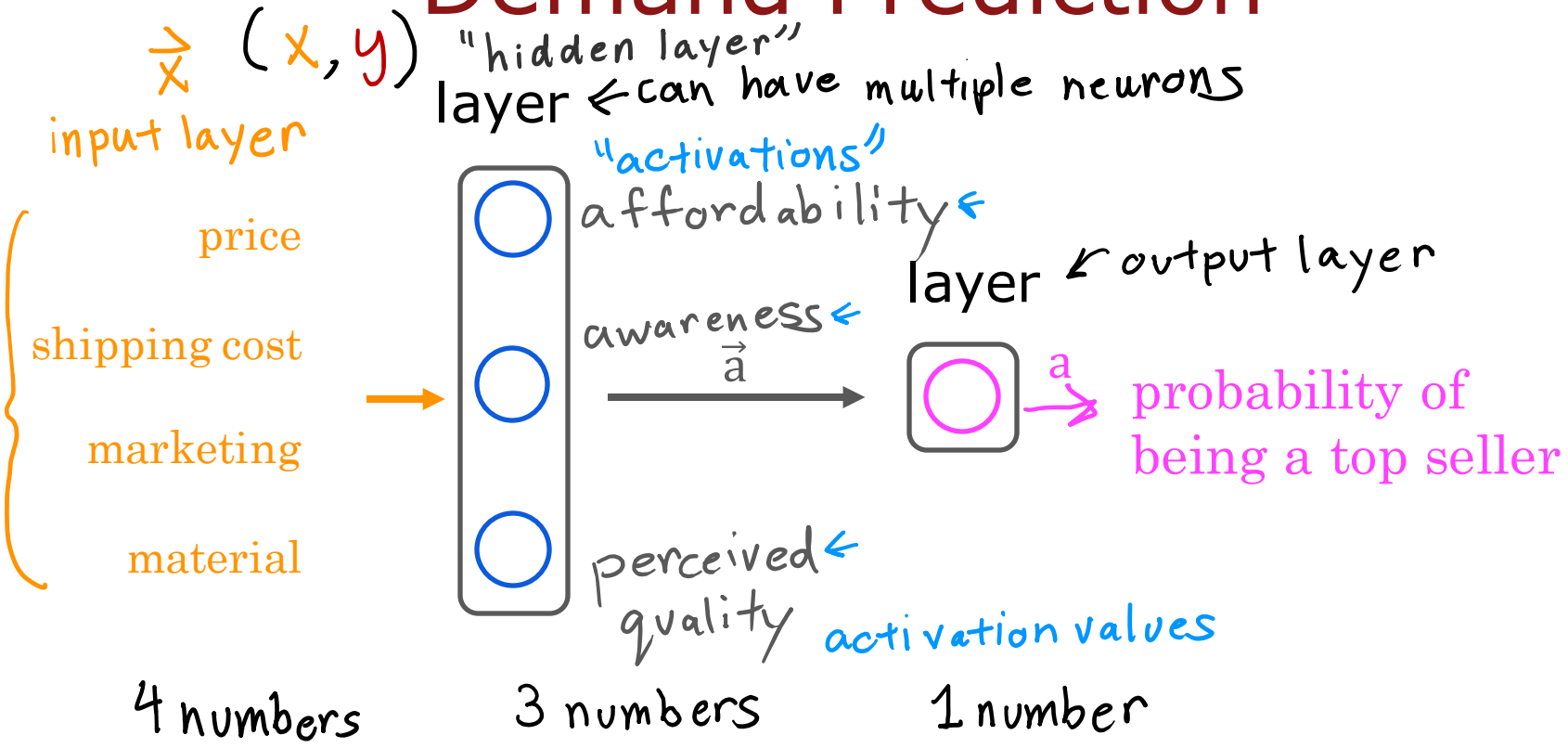
Andrew Ng

Demand Prediction



Andrew Ng

Demand Prediction



Andrew Ng

Demand Prediction

feature engineering

x_1, x_2

$\vec{x}$ (x, y)

input layer

price

shipping cost

marketing

material

"hidden layer"

layer ← can have multiple neurons

"activations"

affordability ←

awareness ←

perceived quality ←

$\vec{a}$

layer ← output layer

a → probability of being a top seller

activation values

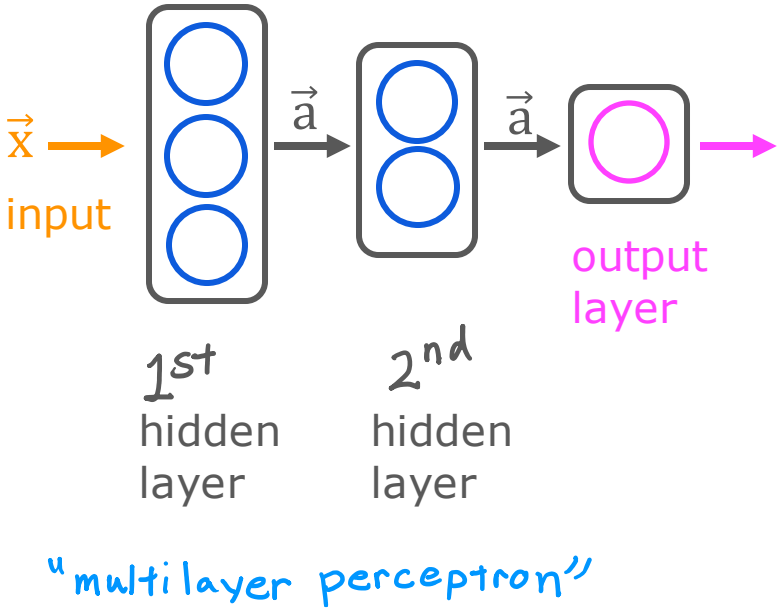
4 numbers

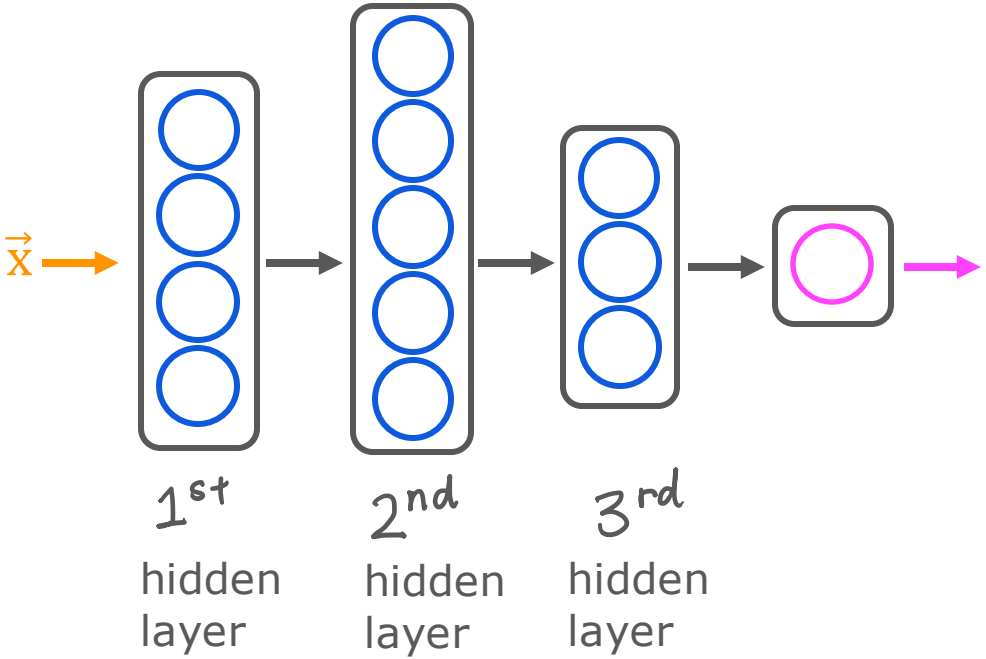
3 numbers

1 number

Andrew Ng

Multiple hidden layers





neural network architecture

Andrew Ng

Face recognition

1000 pixels

1000 pixels

1000 columns

1000 rows

197	185	203	...	...
...	57	64	92	...
⋮				
		...	187	214

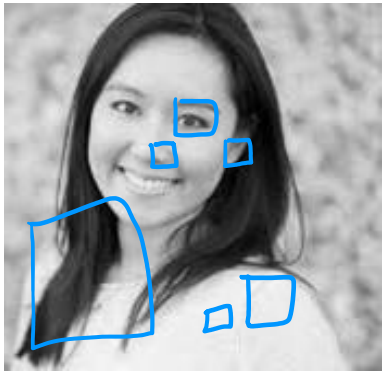
$\vec{x} =$

197
185
203
⋮
57
64
92
⋮
187
214

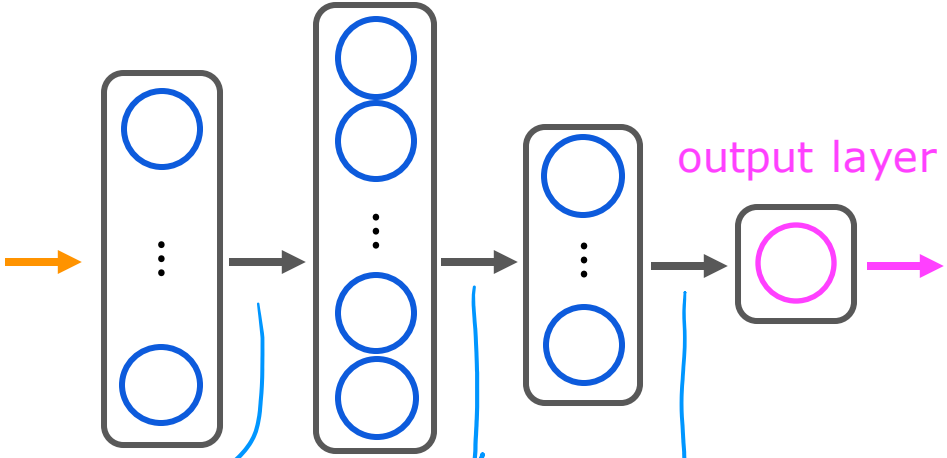
1 million

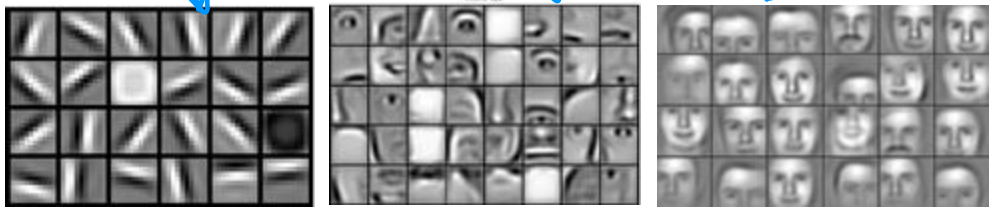
Andrew Ng

Face recognition



$\vec{x}$
input

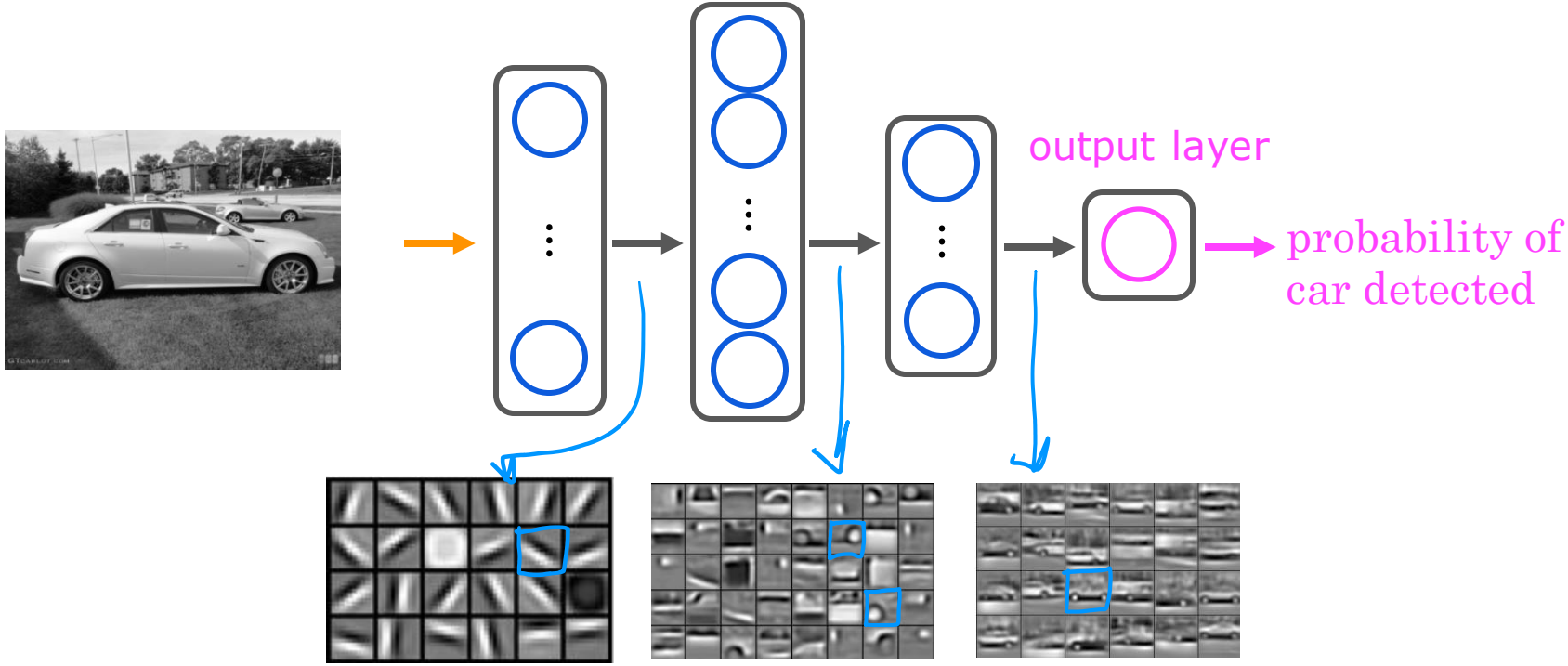




source: Convolutional Deep Belief Networks for Scalable Unsupervised Learning of Hierarchical Representations by Honglak Lee, Roger Grosse, Ranganath Andrew Y. Ng

Andrew Ng

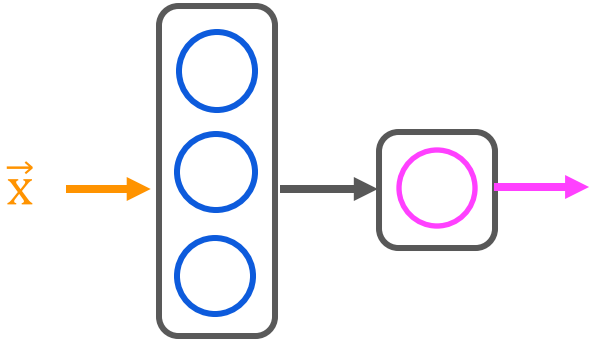
Car classification



source: Convolutional Deep Belief Networks for Scalable Unsupervised Learning of Hierarchical Representations by Honglak Lee, Roger Grosse, Ranganath Andrew Y. Ng

Andrew Ng

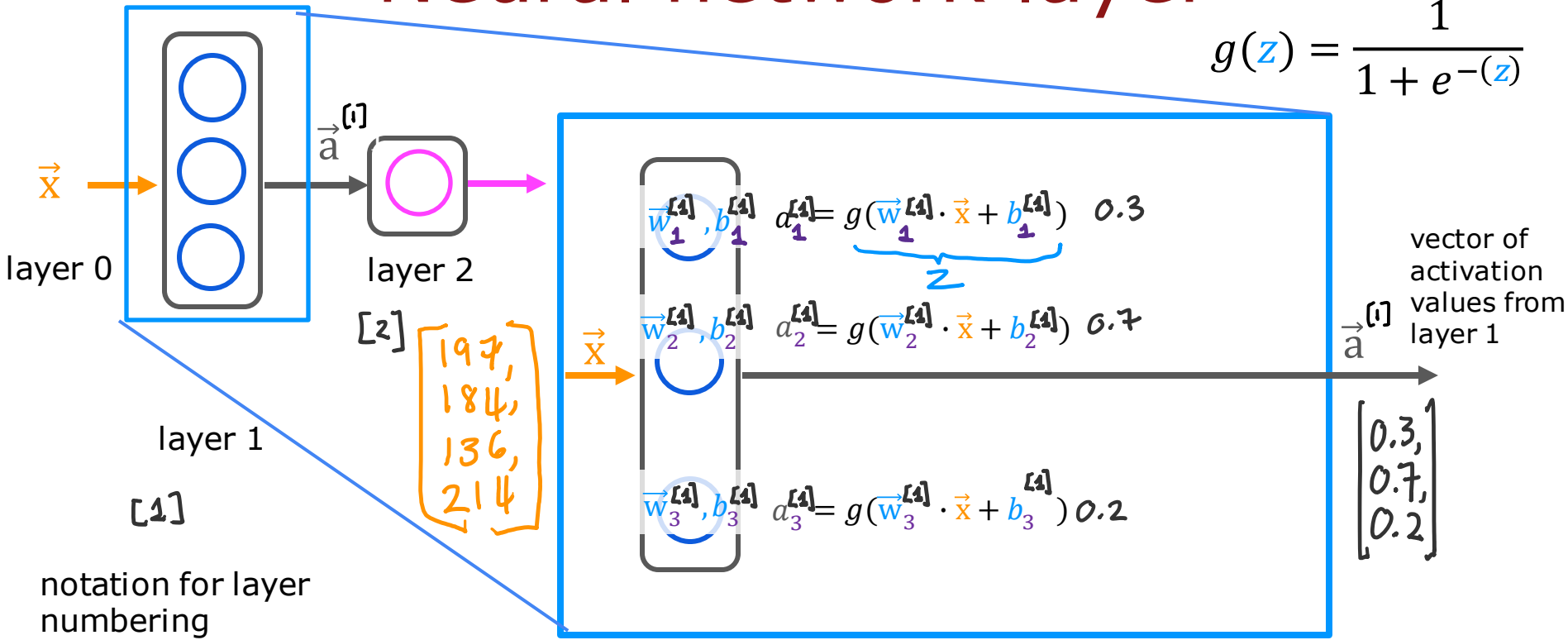
Neural network layer



Andrew Ng

Neural network layer

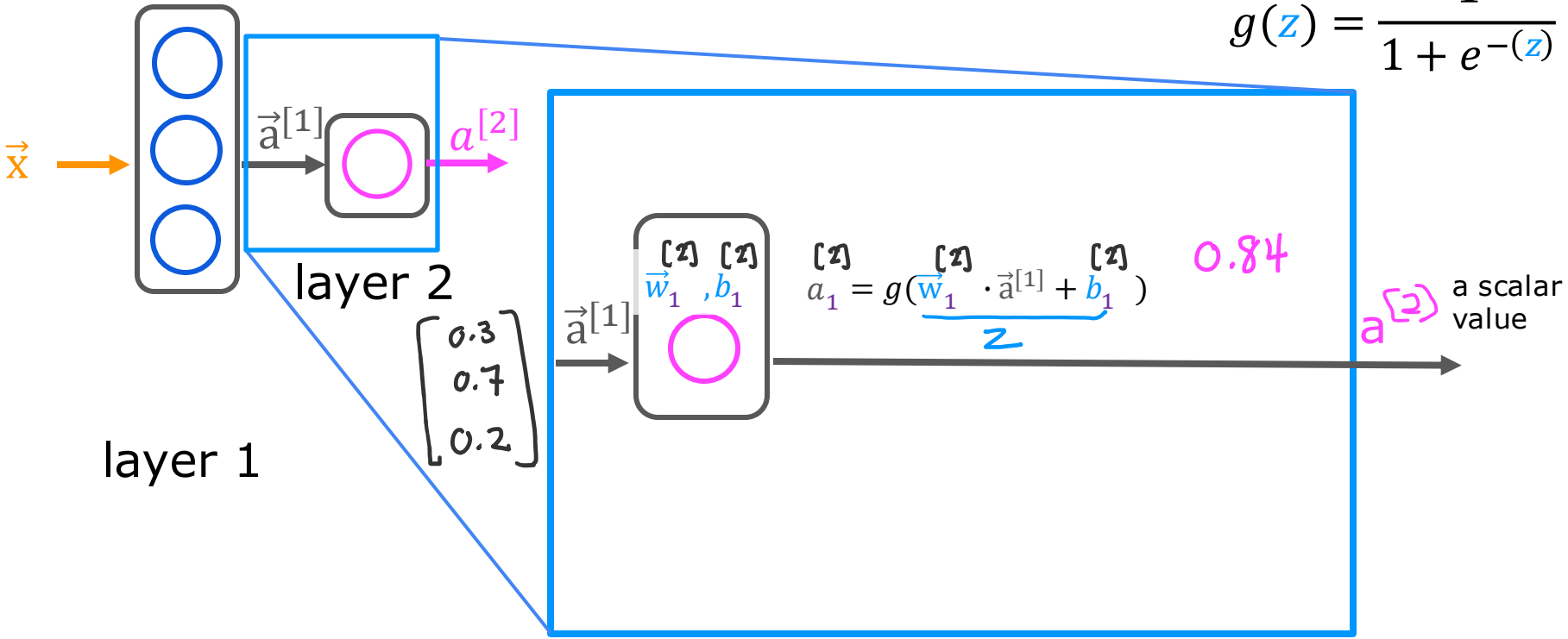
$$g(z) = \frac{1}{1 + e^{-(z)}}$$



Andrew Ng

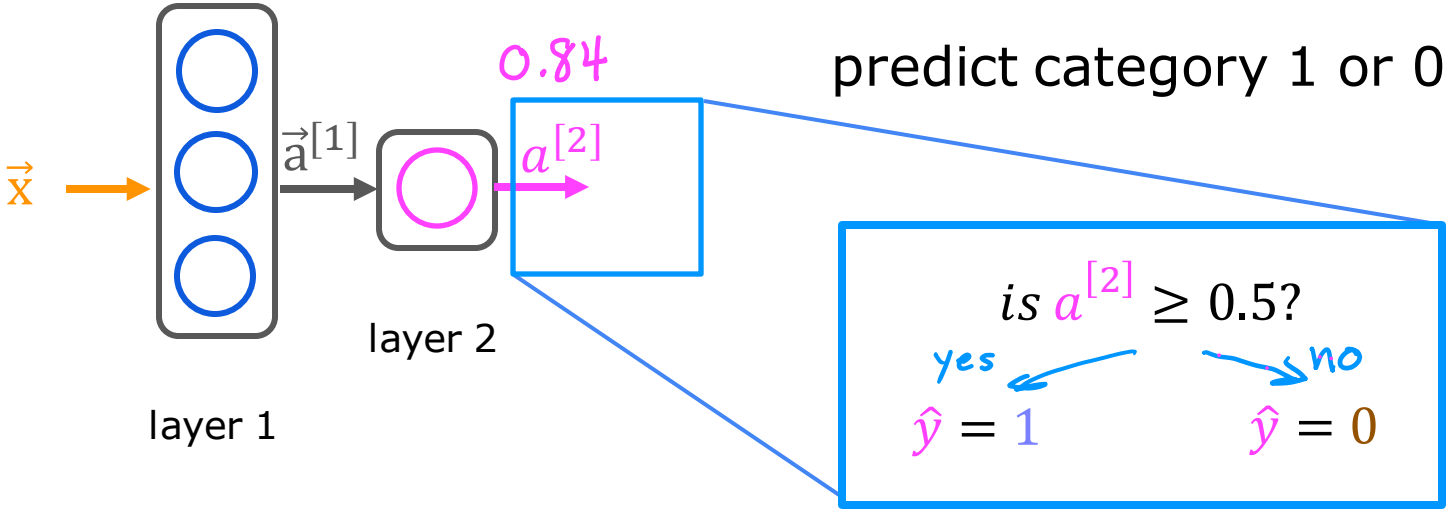
Neural network layer

$$g(z) = \frac{1}{1 + e^{-(z)}}$$



Andrew Ng

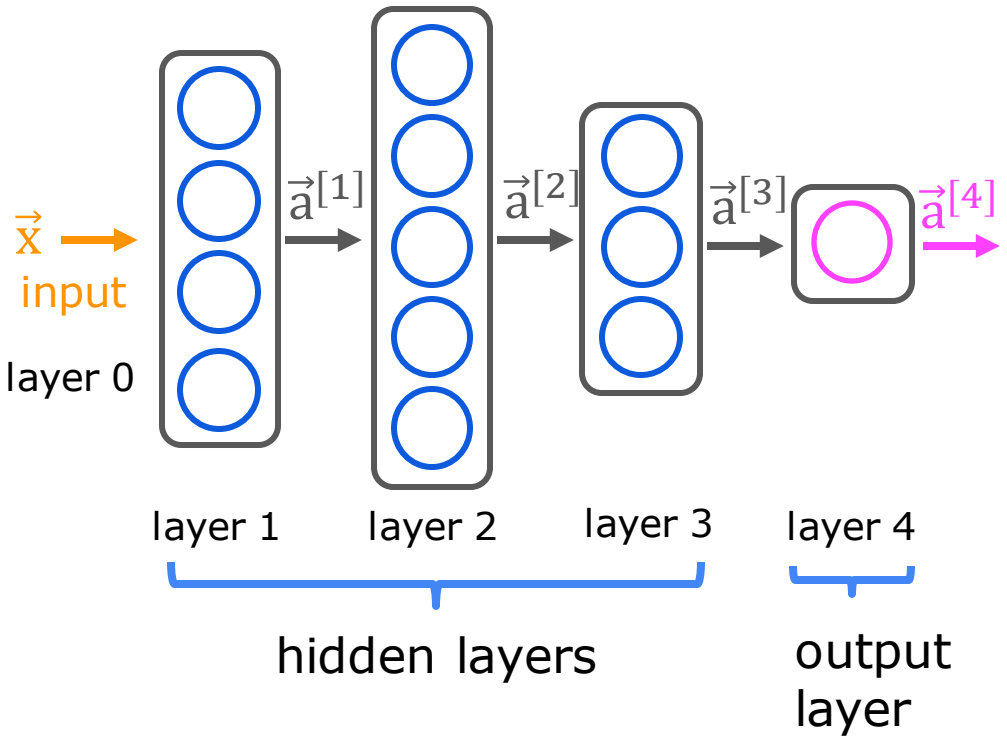
Neural network layer



predict category 1 or 0 (yes/no)

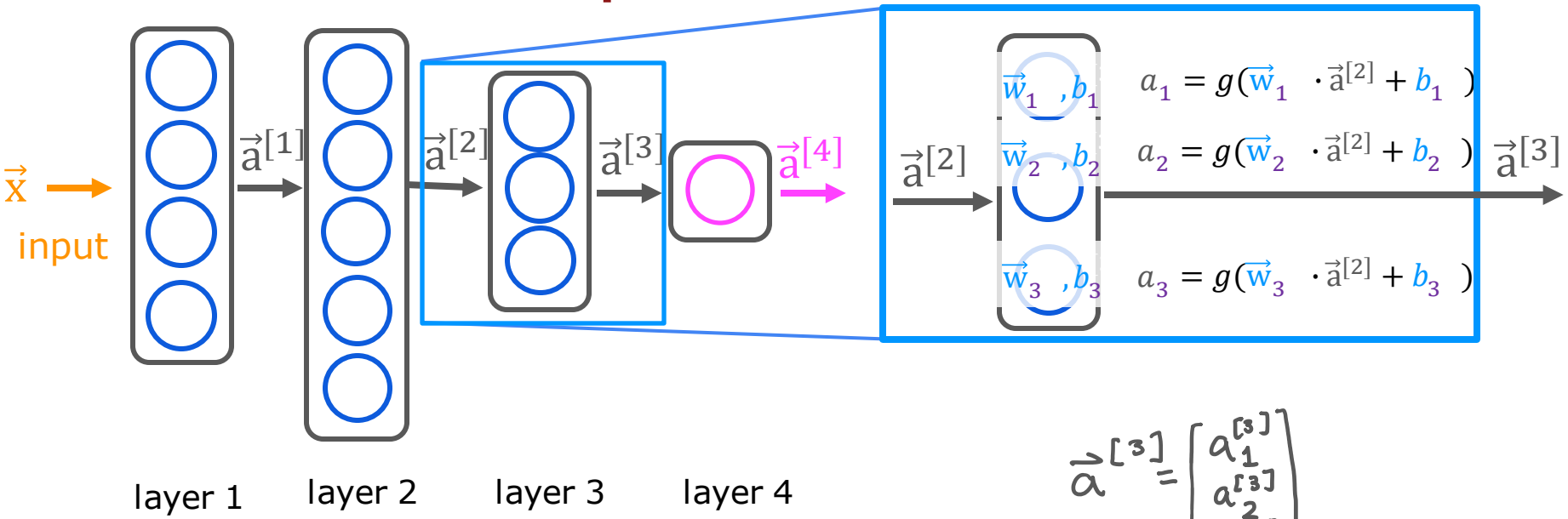
Andrew Ng

More complex neural network



Andrew Ng

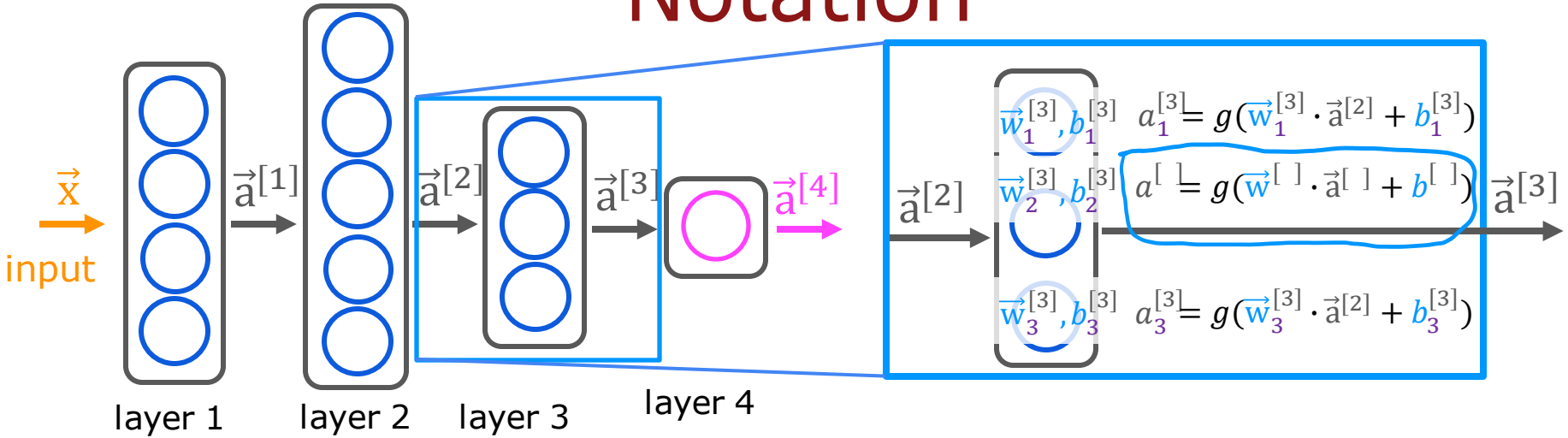
More complex neural network



$$\vec{a}^{[3]} = \begin{bmatrix} a_1^{[3]} \\ a_2^{[3]} \\ a_3^{[3]} \end{bmatrix}$$

Andrew Ng

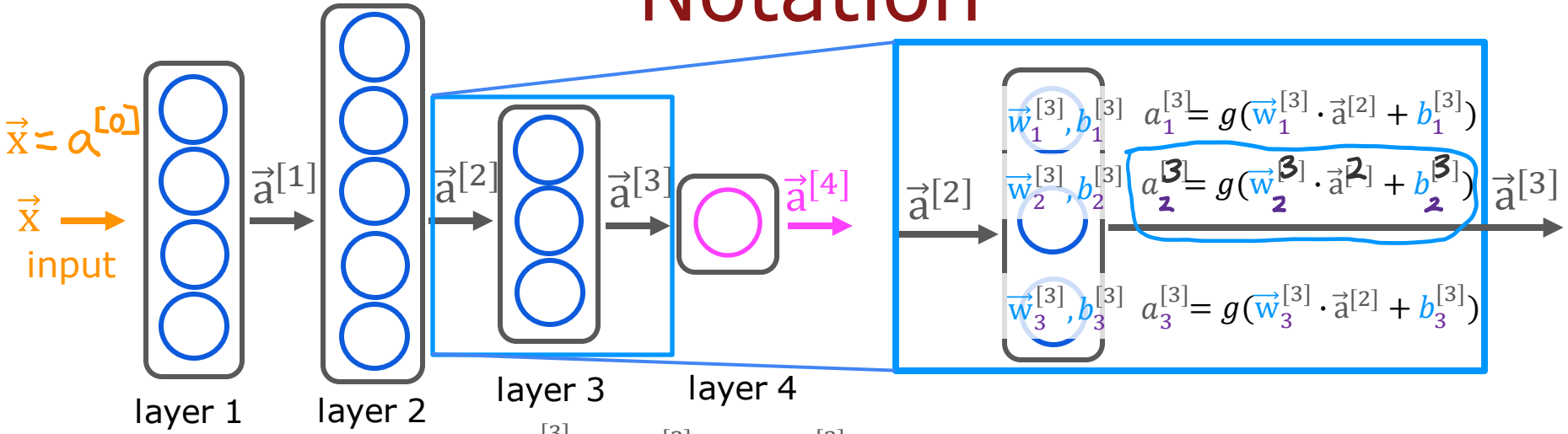
Notation



Question:
Can you fill in the superscripts and subscripts for the second neuron?

Andrew Ng

Notation



$$a_2^{[3]} = g(\vec{w}_2^{[3]} \cdot \vec{a}^{[2]} + b_2^{[3]})$$

Activation value of layer l, unit(neuron) j

output of layer $l-1$ (previous layer)

$$\boxed{a_j^{[l]} = g(\vec{w}_j^{[l]} \cdot \vec{a}^{[l-1]} + b_j^{[l]})}$$

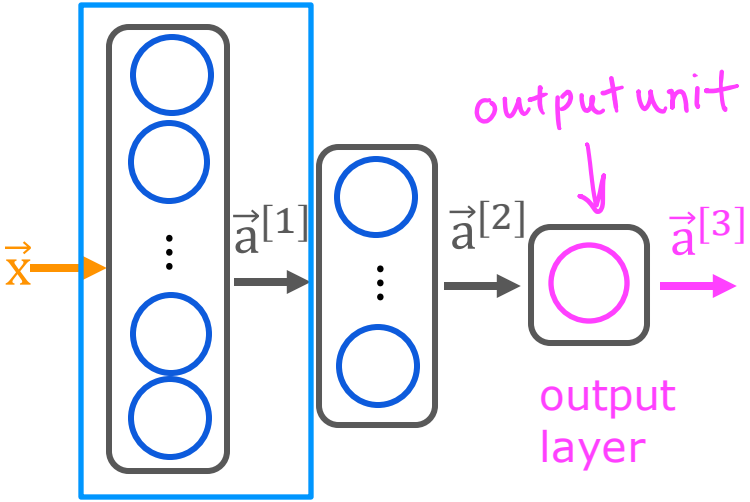
sigmoid

"activation function"

Parameters w & b of layer l, unit j

Andrew Ng

Handwritten digit recognition



probability of being a handwritten '1'

25 units — layer 1

15 units — layer 2

1 unit — layer 3

hidden units (neurons)

Digit images

label 0 1

255	255	255	255	255	255	255	255
255	255	255	0	255	255	255	255
255	255	0	0	255	255	255	255
255	255	255	0	255	255	255	255
255	255	255	0	255	255	255	255
255	255	255	0	255	255	255	255
255	255	0	0	0	255	255	255
255	255	255	255	255	255	255	255

8

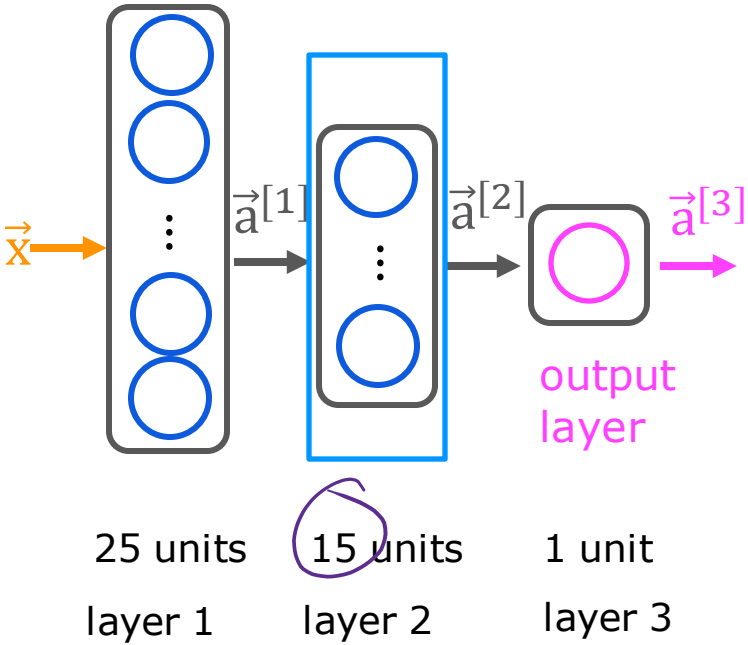
8

$a^{[0]}$

$$\vec{a}^{[1]} = \begin{bmatrix} g(\vec{w}_1^{[1]} \cdot \vec{x} + b_1^{[1]}) \\ \vdots \\ g(\vec{w}_{25}^{[1]} \cdot \vec{x} + b_{25}^{[1]}) \end{bmatrix}$$

Andrew Ng

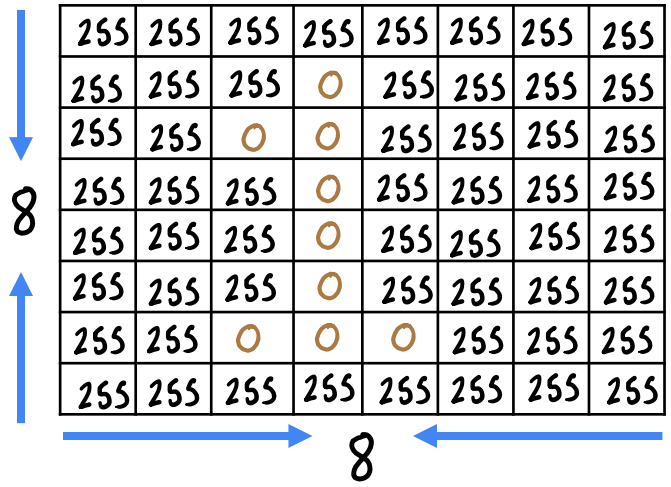
Handwritten digit recognition



probability of being a handwritten '1'

Digit images

label 0 1



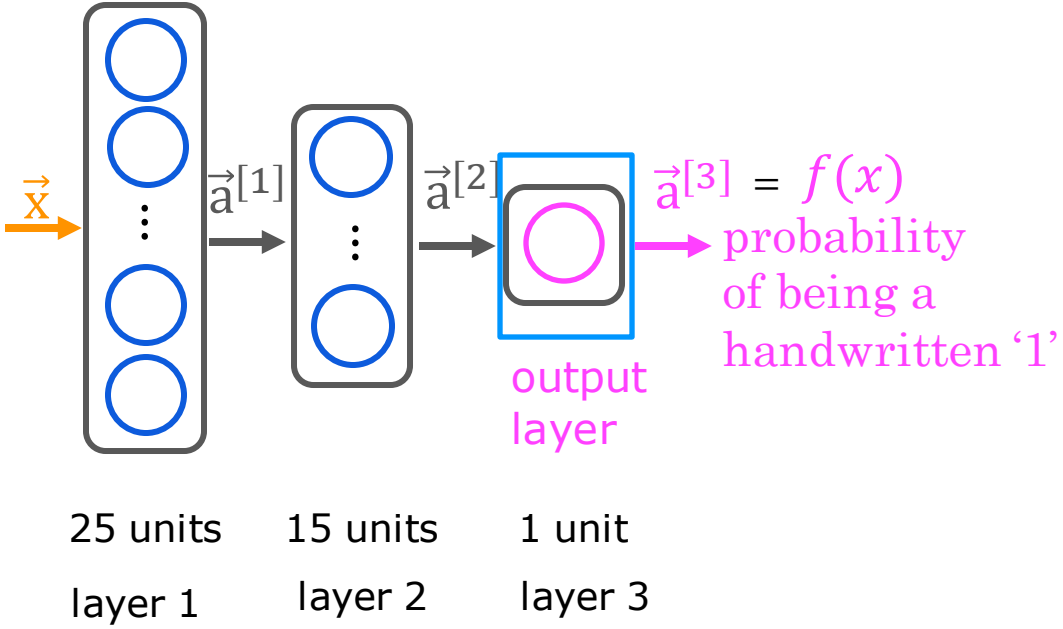
255	255	255	255	255	255	255	255
255	255	255	0	255	255	255	255
255	255	0	0	255	255	255	255
255	255	255	0	255	255	255	255
255	255	255	0	255	255	255	255
255	255	255	0	255	255	255	255
255	255	0	0	0	255	255	255
255	255	255	255	255	255	255	255

$$\vec{a}^{[2]} = \begin{bmatrix} g(\vec{w}_1^{[2]} \cdot \vec{a}^{[1]} + b_1^{[2]}) \\ \vdots \\ g(\vec{w}_{15}^{[2]} \cdot \vec{a}^{[1]} + b_{15}^{[2]}) \end{bmatrix}$$

Andrew Ng

Handwritten digit recognition

forward propagation



$$\vec{a}^{[3]} = \left[g\left(\vec{w}_1^{[3]} \cdot \vec{a}^{[2]} + b_1^{[3]} \right) \right]$$

$$is\ a_1^{[3]} \geq 0.5?$$

yes → $\hat{y} = 1$ image is digit 1

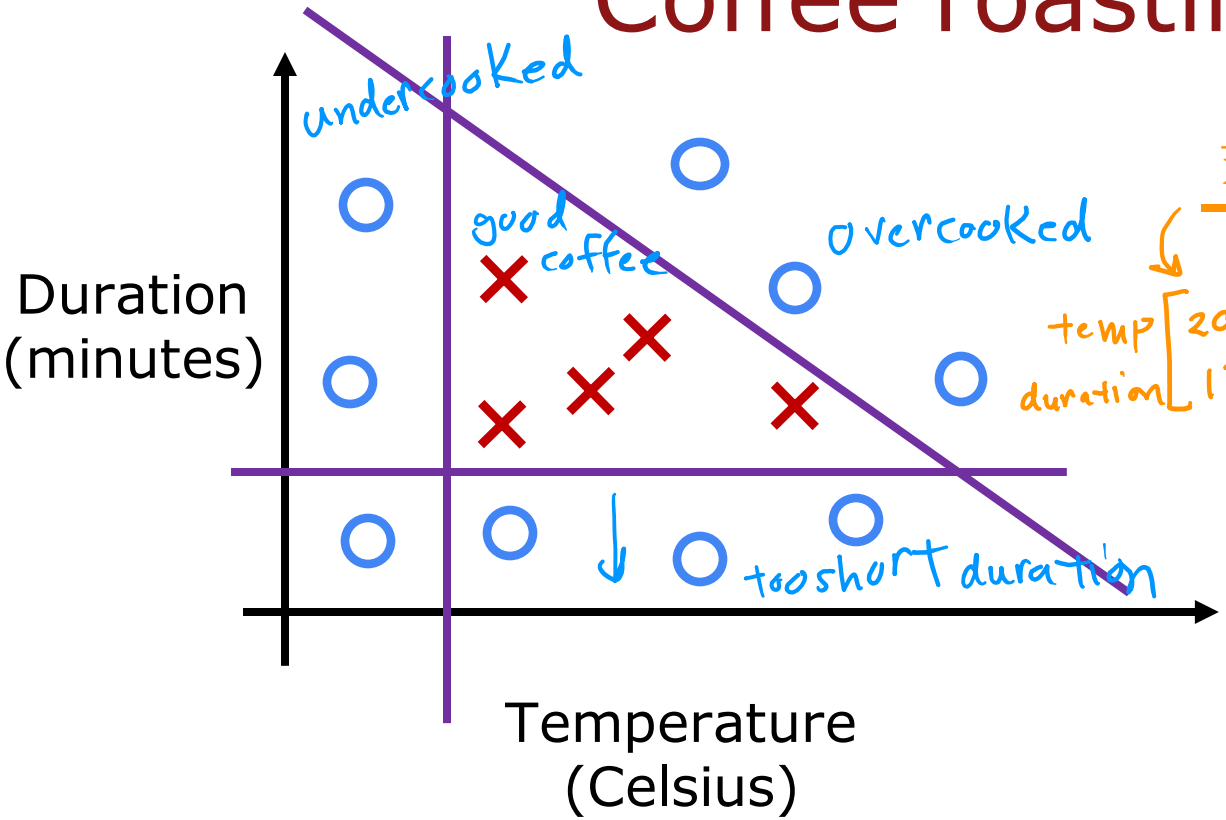
no → $\hat{y} = 0$ image isn't digit 1

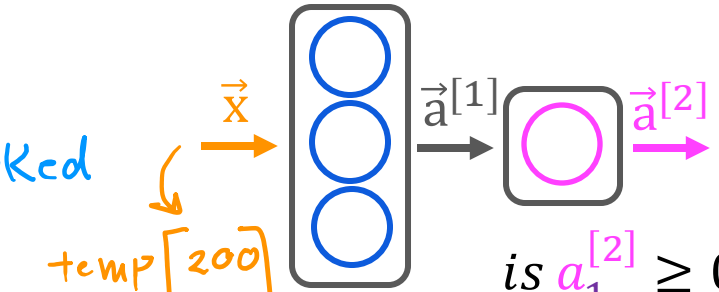
Andrew Ng

TensorFlow implementation

Inference in Code

Coffee roasting



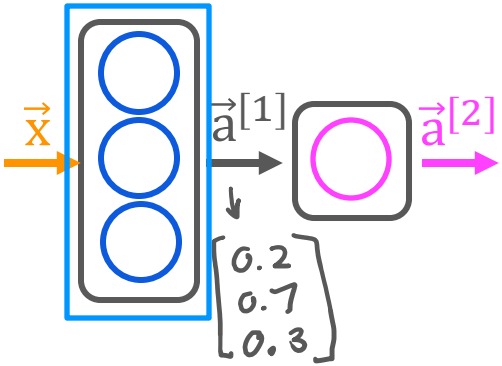


$is\ a_1^{[2]} \geq 0.5?$

yes: $\hat{y} = 1$

no: $\hat{y} = 0$

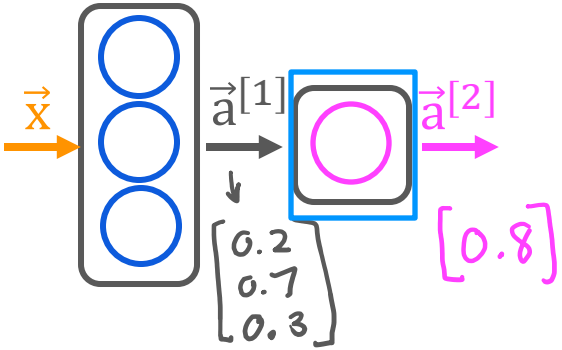
Andrew Ng




```
x = np.array([[200.0, 17.0]])
layer_1 = Dense(units=3, activation='sigmoid')
a1 = layer_1(x)
```

Andrew Ng

Build the model using TensorFlow

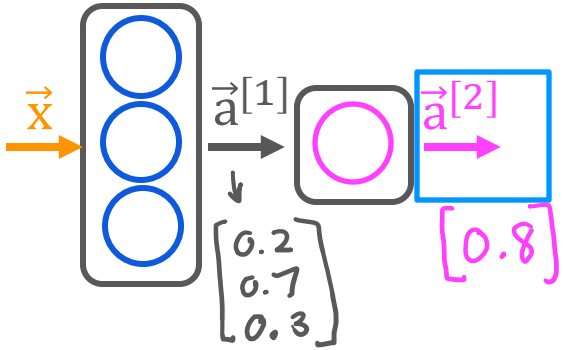



```
x = np.array([[200.0, 17.0]])
layer_1 = Dense(units=3, activation='sigmoid')
a1 = layer_1(x)
```

```
layer_2 = Dense(units=1, activation='sigmoid')
a2 = layer_2(a1)
```

Andrew Ng

Build the model using TensorFlow



$is\ a_1^{[2]} \geq 0.5?$

yes → $\hat{y} = 1$ ✗

no → $\hat{y} = 0$ ○

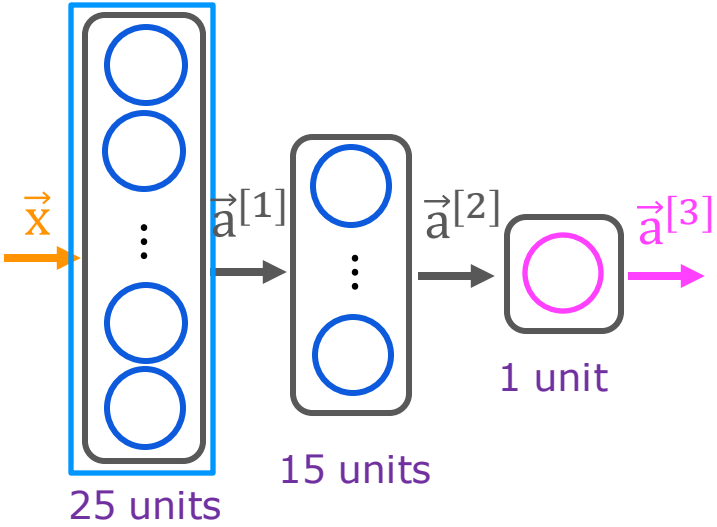
```
x = np.array([[200.0, 17.0]])
layer_1 = Dense(units=3, activation='sigmoid')
a1 = layer_1(x)
```

```
layer_2 = Dense(units=1, activation='sigmoid')
a2 = layer_2(a1)
```

```
if a2 >= 0.5:
  yhat = 1
else:
  yhat = 0
```

Andrew Ng

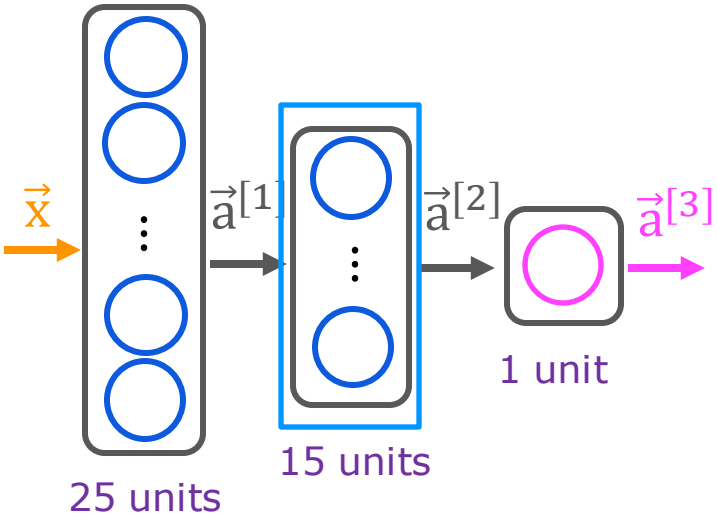
Model for digit classification




```
x = np.array([[0.0,...245,...240...0]])
layer_1 = Dense(units=25, activation='sigmoid')
a1 = layer_1(x)
```

Andrew Ng

Model for digit classification

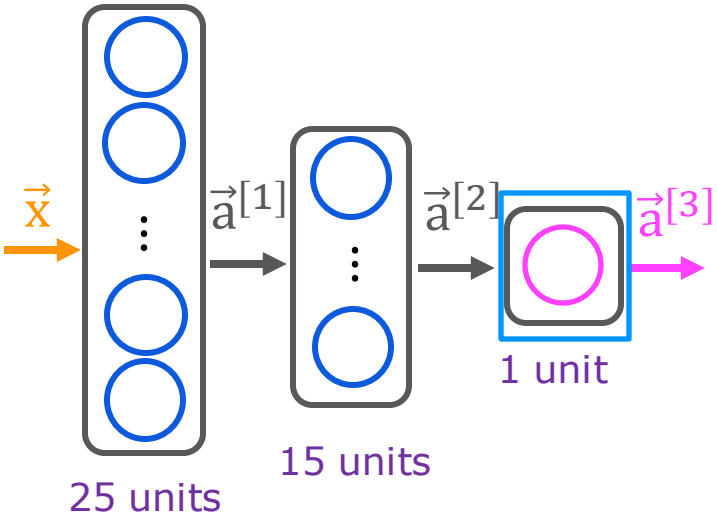



```
x = np.array([[0.0,...245,...240...0]])
layer_1 = Dense(units=25, activation='sigmoid')
a1 = layer_1(x)
```

```
layer_2 = Dense(units=15, activation='sigmoid')
a2 = layer_2(a1)
```

Andrew Ng

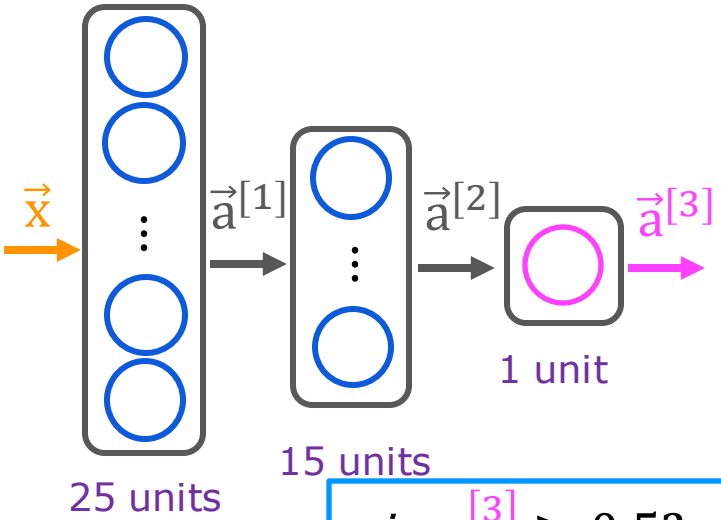
Model for digit classification




```
x = np.array([[0.0,...245,...240...0]])
layer_1 = Dense(units=25, activation='sigmoid')
a1 = layer_1(x)

layer_2 = Dense(units=15, activation='sigmoid')
a2 = layer_2(a1)

layer_3 = Dense(units=1, activation='sigmoid')
a3 = layer_3(a2)
```

Andrew Ng

Model for digit classification




```
x = np.array([[0.0,...245,...240...0]])
layer_1 = Dense(units=25, activation='sigmoid')
a1 = layer_1(x)

layer_2 = Dense(units=15, activation='sigmoid')
a2 = layer_2(a1)

layer_3 = Dense(units=1, activation='sigmoid')
a3 = layer_3(a2)
```

$is\ a_1^{[3]} \geq 0.5?$

$\hat{y} = 1$ ✗ $\hat{y} = 0$ ○

```
if a3 >= 0.5:
  yhat = 1
else:
  yhat = 0
```

Andrew Ng

Feature vectors

temperature (Celsius)	duration (minutes)	Good coffee? (1/0)
200.0	17.0	1
425.0	18.5	0
…	…	…

```
x = np.array([[200.0, 17.0]])
```

```
[[200.0, 17.0]]
```

Why?

Andrew Ng

Note about numpy arrays

3 columns

2 rows $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$

2 x 3 matrix

```
x = np.array([[1, 2, 3],
              [4, 5, 6]])
```

```
[[1, 2, 3],
 [4, 5, 6]]
```

2D array

2 x 3

4 rows $\begin{bmatrix} 0.1 & 0.2 \\ -3 & -4 \\ -.5 & -.6 \\ 7 & 8 \end{bmatrix}$

2 columns

4 x 2 matrix

```
x = np.array([[0.1, 0.2],
              [-3.0, -4.0,],
              [-0.5, -0.6,],
              [7.0, 8.0,]])
```

```
[[0.1, 0.2],
 [-3.0, -4.0,],
 [-0.5, -0.6,],
 [7.0, 8.0,]]
```

4 x 2

1 x 2

2 x 1

Andrew Ng

Note about numpy arrays

```
x = np.array([[200, 17]])
```

$$\begin{bmatrix} 200 & 17 \end{bmatrix}$$

1 x 2

```
x = np.array([[200],
              [17]])
```

$$\begin{bmatrix} 200 \\ 17 \end{bmatrix}$$

2 x 1

```
x = np.array([200,17])
```

1D
"Vector"

Andrew Ng

Feature vectors

temperature (Celsius)	duration (minutes)	Good coffee? (1/0)
200.0	17.0	1
425.0	18.5	0
…	…	…

```
x = np.array([[200.0, 17.0]])
```

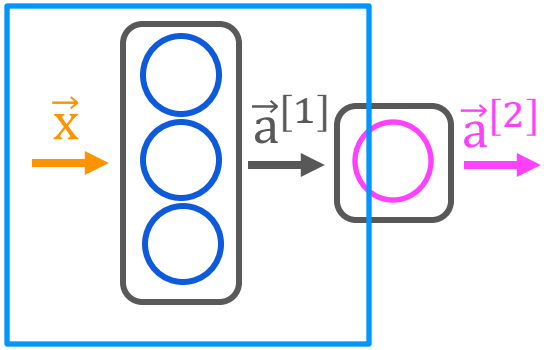
```
[[200.0, 17.0]]
```

1 x 2

$$\begin{bmatrix} 200.0 & 17.0 \end{bmatrix}$$

Andrew Ng

Activation vector




```
x = np.array([[200.0, 17.0]])
layer_1 = Dense(units=3, activation='sigmoid')
a1 = layer_1(x)
```

[[0.2, 0.7, 0.3]] 1 x 3 matrix

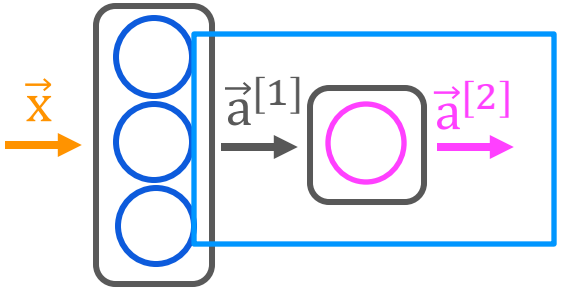
```
tf.Tensor([[0.2 0.7 0.3]], shape=(1, 3), dtype=float32)
```

```
a1.numpy()
```

```
array([[1.4661001, 1.125196 , 3.2159438]], dtype=float32)
```

Andrew Ng

Activation vector




```
layer_2 = Dense(units=1, activation='sigmoid')
a2 = layer_2(a1)
```

[[0.8]]

1 x 1

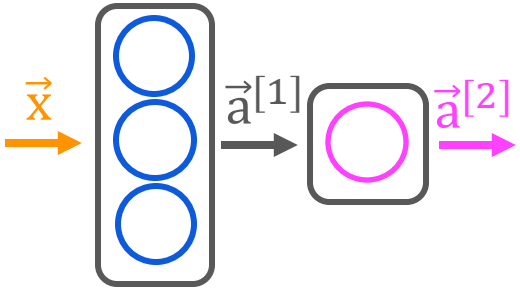
```
tf.Tensor([[0.8]], shape=(1, 1), dtype=float32)
```

```
a2.numpy()
```

```
array([[0.8]], dtype=float32)
```

Andrew Ng

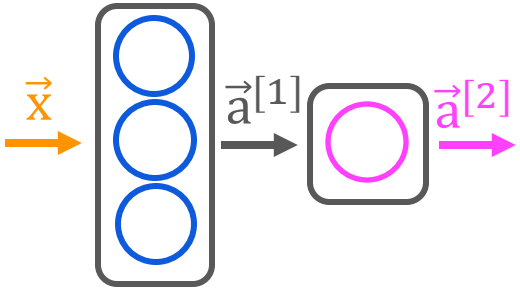
What you saw earlier




```
x = np.array([[200.0, 17.0]])
layer_1 = Dense(units=3, activation="sigmoid")
a1 = layer_1(x)

layer_2 = Dense(units=1, activation="sigmoid")
a2 = layer_2(a1)
```

Andrew Ng

Building a neural network architecture



		y
200	17	1
120	5	0
425	20	0
212	18	1

layer1

```
layer_1 = Dense(units=3, activation="sigmoid")
layer_2 = Dense(units=1, activation="sigmoid")
model = Sequential([layer_1, layer_2])
```

layer2

```
x = np.array([[200.0, 17.0],
              [120.0, 5.0],
              [425.0, 20.0],
              [212.0, 18.0])
```

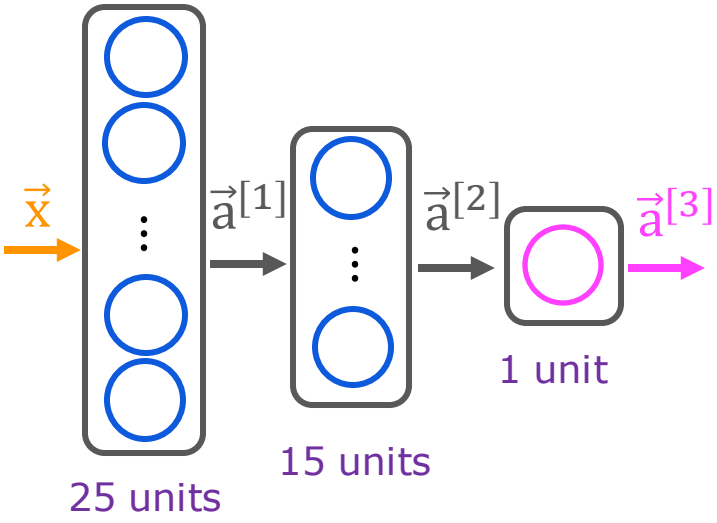
4 x 2

targets

```
y = np.array([1,0,0,1])

model.compile(...)
model.fit(x,y)

model.predict(x_new)
```

Andrew Ng

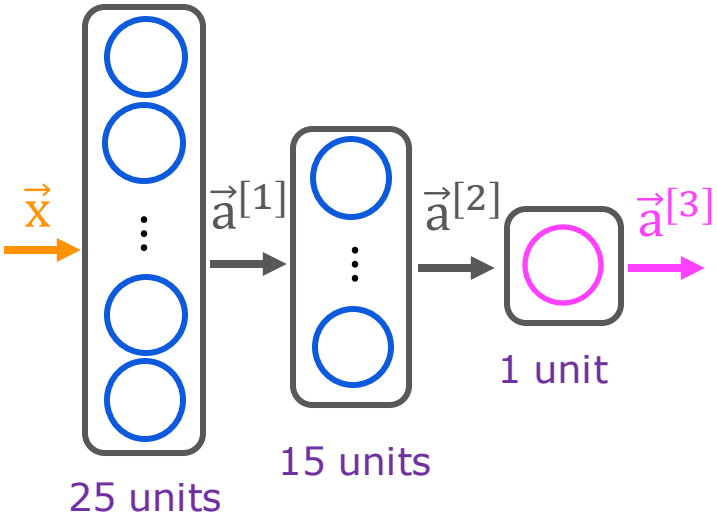
Digit classification model




```
layer_1 = Dense(units=25, activation="sigmoid")
layer_2 = Dense(units=15, activation="sigmoid")
layer_3 = Dense(units=1, activation="sigmoid")

model = Sequential([layer_1, layer_2, layer_3]

model.compile(...)

x = np.array([[0..., 245, ..., 17],
              [0..., 200, ..., 184])
y = np.array([1,0])

model.fit(x,y)

model.predict(x_new)
```

Andrew Ng

Digit classification model



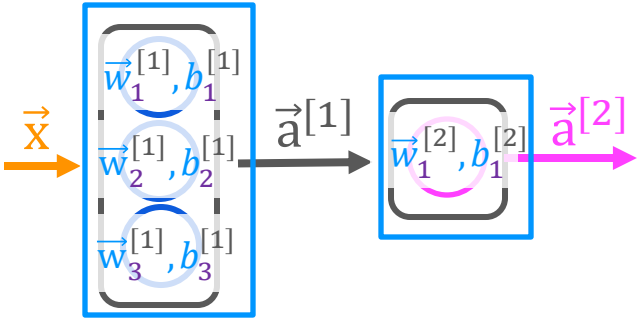

```
model = Sequential([
  Dense(units=25, activation="sigmoid"),
  Dense(units=15, activation="sigmoid"),
  Dense(units=1, activation="sigmoid")])

model.compile(...)

x = np.array([[0..., 245, ..., 17],
              [0..., 200, ..., 184])
y = np.array([1,0])

model.fit(x,y)

model.predict(x_new)
```

Andrew Ng

Neural network implementation in Python

Forward prop in a single layer

forward prop (coffee roasting model)



$$a_1^{[2]} = g(\vec{w}_1^{[2]} \cdot \vec{a}^{[1]} + b_1^{[2]})$$

```
w2_1 = np.array([-7, 8])
b2_1 = np.array([3])
z2_1 = np.dot(w2_1,a1)+b2_1
a2_1 = sigmoid(z1_1)
```

```
x = np.array([200, 17])
```

1D arrays

$$a_1^{[1]} = g(\vec{w}_1^{[1]} \cdot \vec{x} + b_1^{[1]})$$

```
w1_1 = np.array([1, 2])
b1_1 = np.array([-1])
z1_1 = np.dot(w1_1,x)+ b
a1_1 = sigmoid(z1_1)
```

$$a_2^{[1]} = g(\vec{w}_2^{[1]} \cdot \vec{x} + b_2^{[1]})$$

```
w1_2 = np.array([-3, 4])
b1_2 = np.array([1])
z1_2 = np.dot(w1_2,x)+b
a1_2 = sigmoid(z1_2)
```

$$a_3^{[1]} = g(\vec{w}_3^{[1]} \cdot \vec{x} + b_3^{[1]})$$

```
w1_3 = np.array([5, -6])
b1_3 = np.array([2])
z1_3 = np.dot(w1_3,x)+b
a1_3 = sigmoid(z1_3)
```

```
a1   = np.array([a1_1, a1_2, a1_3])
```

Andrew Ng

For loops vs. vectorization

```
x = np.array([200, 17])

W = np.array([[1, -3, 5],
              [-2, 4, -6]])
b = np.array([-1, 1, 2])

def dense(a_in,W,b):
  units = W.shape[1]
  a_out = np.zeros(units)
  for j in range(units):
    w = W[:,j]
    z = np.dot(w,x) + b[j]
    a_out[j] = g(z)
 return a_out
```

[1,0,1]

```
X = np.array([[200, 17]])
```

2D array

```
W = np.array([[1, -3, 5],
              [-2, 4, -6]])
```

same

```
B = np.array([[-1, 1, 2]])
```

1x3 2D array

all 2D arrays

```
def dense(A_in,W,B):
  Z = np.matmul(A_in,W) + B
  A_out = g(Z)
  return A_out
```

vectorized

matrix multiplication

[[1,0,1]]

Andrew Ng

Dot products

example

$$\begin{bmatrix}1\\2\end{bmatrix} \cdot \begin{bmatrix}3\\4\end{bmatrix}$$

$$z = (1 \times 3) + (2 \times 4)$$

$$3 + 8$$

$$11$$

in general

$$\begin{bmatrix}\uparrow\\\vec{a}\\\downarrow\end{bmatrix} \cdot \begin{bmatrix}\uparrow\\\vec{w}\\\downarrow\end{bmatrix}$$

$$z = \vec{a} \cdot \vec{w}$$

transpose

$$\vec{a} = \begin{bmatrix}1\\2\end{bmatrix}$$

$$\vec{a}^T = \begin{bmatrix}1 & 2\end{bmatrix}$$

vector vector multiplication

$$\underset{1\times 2}{\begin{bmatrix}\leftarrow & \vec{a}^T & \rightarrow\end{bmatrix}} \underset{2\times 1}{\begin{bmatrix}\uparrow\\\vec{w}\\\downarrow\end{bmatrix}}$$

$$z = \vec{a}^T \vec{w}$$

equivalent

useful for understanding matrix multiplication

Andrew Ng

Vector matrix multiplication

$$\vec{a} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$\vec{a}^T = [1 \quad 2] \qquad \mathbf{W} = \begin{bmatrix} 3 & 5 \\ 4 & 6 \end{bmatrix} \qquad \mathbf{Z} = \vec{a}^T \mathbf{W} \quad [\leftarrow \quad \vec{a}^T \quad \rightarrow] \begin{bmatrix} \uparrow & \uparrow \\ \vec{w}_1 & \vec{w}_2 \\ \downarrow & \downarrow \end{bmatrix}$$

$$\mathbf{Z} = \begin{bmatrix} \vec{a}^T \vec{w}_1 & a^T \vec{w}_2 \end{bmatrix}$$

$$(1 * 3) + (2 * 4) \qquad\qquad (1 * 5) + (2 * 6)$$

$$3 + 8 \qquad\qquad 5 + 12$$

$$11 \qquad\qquad 17$$

$$\mathbf{Z} = [11 \quad 17]$$

Andrew Ng

matrix matrix multiplication

$$\mathbf{A} = \begin{bmatrix} 1 & -1 \\ 2 & -2 \end{bmatrix}$$

$$\mathbf{A}^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \end{bmatrix} \quad \mathbf{W} = \begin{bmatrix} 3 & 5 \\ 4 & 6 \end{bmatrix} \quad \mathbf{Z} = \mathbf{A}^T\mathbf{W} = \begin{bmatrix} \leftarrow & \vec{a}_1^T & \rightarrow \\ \leftarrow & \vec{a}_2^T & \rightarrow \end{bmatrix} \begin{bmatrix} \uparrow & \uparrow \\ \vec{w}_1 & \vec{w}_2 \\ \downarrow & \downarrow \end{bmatrix}$$

rows columns

row 1 col 1 row 1 col 2

$$= \begin{bmatrix} \vec{a}_1^T \vec{w}_1 & \vec{a}_1^T \vec{w}_2 \\ \vec{a}_2^T \vec{w}_1 & \vec{a}_2^T \vec{w}_2 \end{bmatrix}$$

row 2 col 1 row 2 col 2

$(-1 \times 3) + (-2 \times 4)$

$-3 + -8$

-11

$(-1 \times 5) + (-2 \times 6)$

$-5 + -12$

-17

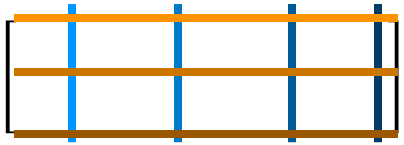
$$= \begin{bmatrix} 11 & 17 \\ -11 & -17 \end{bmatrix}$$

Andrew Ng

Matrix multiplication rules

$$\mathbf{A} = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad \mathbf{A}^T = \begin{array}{c} \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \end{array} \begin{array}{l} \vec{a}_1^{\,T} \\ \vec{a}_2^{\,T} \\ \vec{a}_3^{\,T} \end{array} \mathbf{W} = \begin{array}{c} \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix} \\ \begin{array}{cccc} \vec{w}_1 & \vec{w}_2 & \vec{w}_3 & \vec{w}_4 \end{array} \end{array}$$

$$Z = \mathbf{A}^T \mathbf{W} =$$



Andrew Ng

Matrix multiplication rules

$$\mathbf{A} = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad \mathbf{A}^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \quad \mathbf{W} = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix} \quad \mathbf{Z} = \mathbf{A}^T\mathbf{W} = \begin{bmatrix} 11 & 17 & 23 & 9 \\ -11 & -17 & -23 & -9 \\ 1.1 & 1.7 & 2.3 & 0.9 \end{bmatrix}$$

3 by 4 matrix

$$\vec{a}_1^T \vec{w}_1 = (1 \times 3) + (2 \times 4) = 11$$

row 3 column 2

$$\vec{a}_3^T \vec{w}_2 = (0.1 \times 5) + (0.2 \times 6) = 1.7$$

$$0.5 + 1.2$$

row 2 column 3?

$$\vec{a}_2^T \vec{w}_3 = (-1 \times 7) + (-2 + 8) = -23$$

$$-7 + -16$$

Andrew Ng

Matrix multiplication in NumPy

$$\mathbf{A} = \begin{bmatrix} 1 & -1 & 0.1 \\ 2 & -2 & 0.2 \end{bmatrix} \quad \mathbf{A}^T = \begin{bmatrix} 1 & 2 \\ -1 & -2 \\ 0.1 & 0.2 \end{bmatrix} \quad \mathbf{W} = \begin{bmatrix} 3 & 5 & 7 & 9 \\ 4 & 6 & 8 & 0 \end{bmatrix} \quad \mathbf{Z} = \mathbf{A}^T\mathbf{W} = \begin{bmatrix} 11 & 17 & 23 & 9 \\ -11 & -17 & -23 & -9 \\ 1.1 & 1.7 & 2.3 & 0.9 \end{bmatrix}$$

```
A=np.array([1,-1,0.1],
           [2,-2,0.2]])
```

```
W=np.array([3,5,7,9],
           [4,6,8,0])
```

```
Z = np.matmul(AT,W)
```

or

```
Z = AT @ W
```

```
AT=np.array([1,2],
            [-1,-2],
            [0.1,0.2])
```

```
AT=A.T
```

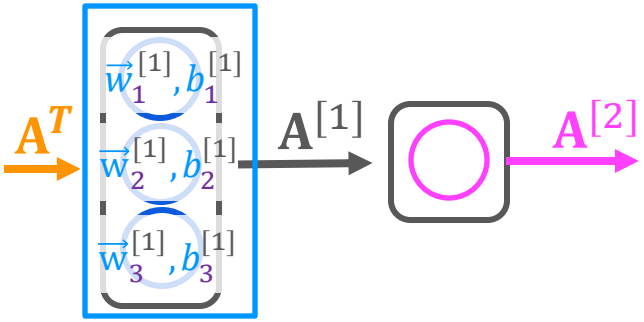
transpose

result

```
[[11,17,23,9],
 [-11,-17,-23,-9],
 [1.1,1.7,2.3,0.9]
]
```

Andrew Ng

Dense layer vectorized



$$\mathbf{A}^T = \begin{bmatrix} 200 & 17 \end{bmatrix}$$

1 x 2

$$\mathbf{W} = \begin{bmatrix} \boxed{\begin{matrix} 1 \\ -2 \end{matrix}} & \boxed{\begin{matrix} -3 \\ 4 \end{matrix}} & \boxed{\begin{matrix} 5 \\ -6 \end{matrix}} \end{bmatrix}$$

2 x 3

$$\vec{\mathbf{b}} = \begin{bmatrix} -1 & 1 & 2 \end{bmatrix}$$

1 x 3

$$\mathbf{Z} = \mathbf{A}^T\mathbf{W} + \mathbf{B}$$

$$\begin{array}{c} \begin{bmatrix} \boxed{165} & \boxed{-531} & \boxed{900} \end{bmatrix} \\ z_1^{[1]} \quad z_2^{[1]} \quad z_3^{[1]} \end{array}$$

$$\mathbf{A} = g(\mathbf{Z})$$

$$\begin{bmatrix} 1 & 0 & 1 \end{bmatrix}$$

```
AT = np.array([[200, 17]])
W = np.array([[1, -3, 5],
              [-2, 4, -6])
b = np.array([[-1, 1, 2]])

def dense(AT,W,b,g):
  z = np.matmul(AT,W) + b
  a_out = g(z)
  return a_out
```

[[1,0,1]]

Andrew Ng